Test Doubles

Os **test doubles** (*dublês de teste*) são objetos ou funções substitutas usados em testes para ocupar o lugar de dependências reais do sistema. A ideia central é simples: quando o componente testado depende de algo externo ou secundário, muitas vezes não é desejável usar a dependência concreta no teste. Em vez disso, utiliza-se um substituto controlável.

O termo **test double** é um termo genérico. Assim como, no cinema, um dublê substitui um ator em certas cenas, no teste um *double* substitui uma dependência real dentro de um software. Dentro dessa categoria geral, surgem tipos mais específicos, com finalidades distintas:

- **Dummy**
- **Stub**
- **Spy**
- **Mock**
- **Fake**

Essa distinção é importante porque, em muitos contextos, tudo acaba sendo chamado genericamente de “mock”, o que empobrece a precisão conceitual.

1. Ideia fundamental dos test doubles

Suponha uma função ou classe que dependa de:

- banco de dados
- API externa
- serviço de e-mail
- relógio do sistema
- gerador de identificadores
- repositório
- serviço de autenticação

Em um teste, frequentemente você não quer:

- acessar rede
- gravar no banco
- enviar e-mail real
- depender da hora atual
- depender de efeitos colaterais imprevisíveis

Nesse cenário, o *test double* serve para:

- **isolar** o componente testado
- **controlar** entradas indiretas
- **observar** interações
- **simular** comportamentos relevantes
- **reduzir** custo, lentidão e instabilidade

Em síntese, test doubles ajudam a tornar o teste mais:

- determinístico
- rápido
- focado
- legível

2. SUT

Antes de estudar os tipos de doubles, convém fixar um termo muito usado na literatura e na prática.

> **SUT** significa **System Under Test**.

Em contexto didático, pode ser entendido como:

- o componente sob teste
- a unidade efetivamente testada naquele caso
- o objeto cujo comportamento queremos verificar

Exemplos:

- se o teste é sobre a classe `ApproveTripRequest`, então o **SUT** é `ApproveTripRequest`
- se o teste é sobre a função `validateCpf`, então o **SUT** é `validateCpf`
- se o teste é sobre o método `calculateTotal`, então esse método, dentro do contexto do teste, é o **SUT**

2.2 Por que `SUT` é um bom nome?

O nome `SUT` é muito bom por cinco razões:

1. **É semântico**
   - ele deixa explícito qual objeto está no centro do teste
2. **É estável**
   - mesmo que a classe mude de nome durante refatoração, o papel daquele objeto no teste continua sendo o mesmo: ele é o componente sob teste
3. **Evita ruído**
   - em vez de criar nomes artificiais como `approveTripRequestUnderTestInstance`, usa-se simplesmente `sut`
4. **Padroniza a leitura dos testes**
   - ao bater o olho em `sut.execute(...)`, já se sabe que a chamada principal está sendo feita sobre o componente que o teste deseja avaliar
5. **Favorece a organização do arranjo de teste**
   - quando usamos uma função como `makeSut()`, fica natural separar:
     - o **SUT**
     - suas dependências
     - os test doubles usados para montá-lo

Por isso, `sut` se tornou um nome tradicional e extremamente expressivo em testes automatizados.

3. Existe hierarquia entre dummy, stub, spy, mock e fake?

Não existe uma **hierarquia rígida, canônica e universal** entre esses tipos. A hierarquia realmente segura é apenas esta:

- **Test Double**
  - **Dummy**
  - **Stub**
  - **Spy**
  - **Mock**
  - **Fake**

Ou seja, todos são **espécies do gênero test double**.

O que melhor os distingue não é um grau de “evolução”, mas sim o **papel predominante** que desempenham no teste.

Por isso, não é correto pensar assim:

- dummy < stub < spy < mock < fake

Essa suposta escala não se sustenta conceitualmente.

O mais adequado é pensar assim:

- **Dummy**: ocupa espaço
- **Stub**: fornece dados
- **Spy**: registra interações
- **Mock**: impõe expectativas de interação
- **Fake**: substitui por implementação funcional simplificada

Além disso, um mesmo objeto pode **acumular mais de um papel**. Um double pode, por exemplo, devolver dados e também registrar chamadas. Ainda assim, para fins conceituais, convém classificá-lo pela **finalidade predominante**.

4. Tabela comparativa

A tabela abaixo organiza os tipos de doubles por **papel predominante no teste**, e não por hierarquia linear.

Test double	Finalidade principal	Participa da lógica do teste?	Verifica interação?	Comportamento típico	Exemplo simples	Confusão comum
Dummy	Apenas preencher parâmetro ou assinatura	Não	Não	É passado ao SUT, mas não influencia o cenário	passar um `logger` vazio só para satisfazer o construtor	chamar de stub algo que não responde nem registra nada

Test double	Finalidade principal	Participa da lógica do teste?	Verifica interação?	Comportamento típico	Exemplo simples	Confusão comum
Stub	Fornecer respostas controladas ao SUT	Sim	Não, em regra	Devolve valores prontos, previsíveis e relevantes ao cenário	repositório que sempre retorna uma solicitação pendente	confundir com fake ou chamar genericamente de mock
Spy	Registrar como a dependência foi usada	Sim	Sim	Armazena chamadas, argumentos e contagens para inspeção posterior	notificador que guarda os e-mails enviados	achar que spy já impõe expectativa prévia como mock
Mock	Validar expectativas explícitas de interação	Sim	Sim, de forma central	O teste define o que deve acontecer e falha se isso não ocorrer	dependência configurada para exigir `save(request)` uma vez	usar “mock” como rótulo para qualquer double
Fake	Substituir a dependência por uma implementação funcional simplificada	Sim	Pode ou não	Funciona de verdade, mas de forma simplificada e local ao teste	repositório em memória em vez de banco real	tratá-lo apenas como um stub sofisticado

Leitura resumida:

- **Dummy**: só ocupa lugar
- **Stub**: devolve respostas
- **Spy**: registra chamadas
- **Mock**: cobra expectativas
- **Fake**: funciona de verdade, mas de forma simplificada

5. Dummy

5.1 Definição

Um **dummy** é um objeto meramente colocado para preencher uma assinatura, mas que **não participa de fato do teste**.

Ele existe apenas porque a função ou o construtor exige um argumento, embora esse argumento não seja relevante naquele caso de teste.

5.2 Característica essencial

O dummy:

- é passado para o SUT
- não é realmente usado no comportamento testado
- não precisa ter lógica útil

5.3 Exemplo conceitual

Imagine:

```
class CreateUser {
  constructor(
    private readonly userRepository: UserRepository,
    private readonly mailService: MailService,
  ) {}
}
```

Agora suponha que, em certo teste, você quer validar apenas uma regra que falha **antes** de qualquer uso do `mailService`. Ainda assim, o construtor exige esse parâmetro. Você pode passar um dummy.

5.4 Exemplo

```
interface MailService {
  sendWelcomeEmail(email: string): Promise<void>;
}

class DummyMailService implements MailService {
  async sendWelcomeEmail(_email: string): Promise<void> {
    throw new Error("Should not be called");
  }
}
```

Neste exemplo, o **test double** é `DummyMailService`.

Se o fluxo chegar até `sendWelcomeEmail`, o teste provavelmente está mal montado ou o comportamento do SUT não é o esperado para aquele cenário.

5.5 Quando usar

Use dummy quando:

- a dependência é obrigatória na assinatura
- ela não interfere no cenário testado
- você só precisa “preencher espaço”

5.6 Risco didático importante

Se um dummy começa a ser consultado, responder dados ou registrar chamadas, ele já deixou de ser dummy.

6. Stub

6.1 Definição

Um **stub** é um double usado para **fornecer respostas controladas** ao SUT.

Ele é usado quando o teste precisa que a dependência retorne valores específicos, para conduzir a execução por determinado caminho.

6.2 Característica essencial

O stub:

- responde com dados previamente definidos
- ajuda a montar o cenário
- normalmente não serve para verificar interações

Em outras palavras, o foco do stub é:

> **“quando chamado, devolva isto”**

6.3 Exemplo conceitual

Suponha um caso de uso que consulta um usuário por e-mail:

```
interface UserRepository {
  findByEmail(email: string): Promise<User | null>;
}
```

Se você quer testar o comportamento quando o usuário **já existe**, pode usar um stub que devolve um usuário.

6.4 Exemplo

```
type User = {
  id: string;
  email: string;
};

interface UserRepository {
  findByEmail(email: string): Promise<User | null>;
}

class UserRepositoryStub implements UserRepository {
  async findByEmail(_email: string): Promise<User | null> {
    return {
      id: "user-1",
      email: "john@example.com",
    };
  }
}
```

Neste exemplo, o **test double** é `UserRepositoryStub`.

O seu papel é devolver ao SUT um usuário já existente, de modo controlado.

6.5 Quando usar

Use stub quando:

- o SUT depende de dados vindos de outra camada
- você precisa controlar o retorno da dependência
- o interesse do teste está no resultado final do SUT, não nas chamadas em si

6.6 Observação importante

Se, além de responder dados, você começa a registrar quantas vezes foi chamado ou com quais argumentos, esse double passa a ter comportamento de spy.

7. Spy

7.1 Definição

Um **spy** é um double que **registra informações sobre como foi utilizado**.

Ele permite observar interações do SUT com a dependência.

7.2 Característica essencial

O spy responde a perguntas como:

- a dependência foi chamada?
- com quais argumentos?
- quantas vezes?

O foco do spy é **observação posterior**.

7.3 Exemplo conceitual

Imagine um serviço que cria um usuário e, ao final, envia um e-mail de boas-vindas.

Você quer verificar se o envio do e-mail foi solicitado corretamente.

7.4 Exemplo

```
interface MailService {
  sendWelcomeEmail(email: string): Promise<void>;
}

class MailServiceSpy implements MailService {
  public readonly sentEmails: string[] = [];

  async sendWelcomeEmail(email: string): Promise<void> {
    this.sentEmails.push(email);
  }
}
```

Neste exemplo, o **test double** é `MailServiceSpy`.

No teste, a verificação costuma aparecer assim:

```
expect(mailServiceSpy.sentEmails).toEqual(["john@example.com"]);
```

7.5 Quando usar

Use spy quando:

- você quer verificar efeito indireto
- o comportamento relevante é uma interação com outra dependência
- deseja inspecionar chamadas depois da execução

7.6 Distinção importante em relação ao stub

- **Stub**: fornece respostas
- **Spy**: registra chamadas

Um mesmo double pode acumular ambos os papéis, mas conceitualmente convém distinguir a finalidade predominante.

8. Mock

8.1 Definição

Um **mock** é um double orientado à **verificação de expectativas de interação**.

Ele é usado quando o teste quer afirmar que certas chamadas devem ocorrer de forma específica.

8.2 Característica essencial

O mock não é apenas um objeto que registra chamadas. Ele participa da lógica do teste como um elemento cuja interação esperada foi previamente definida.

A ênfase do mock é:

> **“esta dependência deve ser chamada desta forma”**

8.3 Perspectiva conceitual

Em muitos ambientes, especialmente com bibliotecas de mocking, o mock:

- já nasce configurado com expectativas
- pode falhar o teste se a expectativa não for satisfeita
- enfatiza comportamento observável entre objetos

8.4 Exemplo conceitual

Suponha um caso de uso que salva um pedido e notifica o cliente. Você quer afirmar que o serviço de notificação foi chamado exatamente uma vez com o e-mail correto.

Com biblioteca de teste, isso frequentemente aparece assim:

```
expect(notificationService.send).toHaveBeenCalledWith("john@example.com");
```

Nesse contexto, o **test double** é a implementação simulada de `notificationService.send` configurada pela biblioteca.

8.5 Diferença entre spy e mock

Na prática cotidiana, a fronteira entre **spy** e **mock** é frequentemente borrada por ferramentas. Porém, conceitualmente:

- **Spy**: registra o que aconteceu, e o teste depois consulta esse registro
- **Mock**: expressa e verifica expectativas de interação de forma mais explícita e central

Uma formulação útil é esta:

- **spy observa**
- **mock exige**

8.6 Quando usar

Use mock quando:

- a interação com a dependência é parte central do comportamento
- você quer verificar protocolo de colaboração entre objetos
- o teste é mais comportamental do que orientado apenas ao estado final

8.7 Cuidado pedagógico

O uso excessivo de mocks pode levar a testes:

- frágeis
- excessivamente acoplados à implementação interna
- difíceis de manter durante refatorações

Isso ocorre quando o teste passa a descrever detalhes de colaboração que não são essenciais para o comportamento de negócio.

9. Fake

9.1 Definição

Um **fake** é uma implementação alternativa, funcional e simplificada de uma dependência real.

Diferentemente do stub ou do dummy, o fake geralmente possui uma lógica própria, ainda que reduzida, adequada ao teste.

9.2 Característica essencial

O fake:

- funciona de verdade
- mas de forma simplificada
- substitui uma dependência complexa por uma implementação mais leve

9.3 Exemplo clássico

Um repositório em memória no lugar de um banco de dados real.

9.4 Exemplo

```
type User = {
  id: string;
  email: string;
};

interface UserRepository {
  save(user: User): Promise<void>;
  findByEmail(email: string): Promise<User | null>;
}

class InMemoryUserRepository implements UserRepository {
  private readonly items: User[] = [];

  async save(user: User): Promise<void> {
    this.items.push(user);
  }

  async findByEmail(email: string): Promise<User | null> {
    return this.items.find((user) => user.email === email) ?? null;
  }
}
```

Neste exemplo, o **test double** é `InMemoryUserRepository`.

Ele não apenas devolve um valor fixo. Ele realmente mantém estado e implementa comportamento útil.

9.5 Quando usar

Use fake quando:

- uma dependência real é cara, lenta ou complexa
- você quer uma implementação funcional mais simples
- o teste se beneficia de algo mais realista que um stub

9.6 Vantagem importante

Fakes frequentemente produzem testes mais naturais e menos acoplados que mocks em excesso.

9.7 Cuidado

Um fake não deve reproduzir toda a complexidade da infraestrutura real. Se ele se torna complexo demais, passa a ser difícil de confiar e manter.

10. Não confundir tipo de test double com tipo de teste

Esse é um ponto muito importante.

O fato de um teste usar stub, spy, mock ou fake **não define, por si só, a natureza do teste**.

Um teste pode usar doubles e ainda assim ser:

- unitário
- de integração
- mais sociável
- mais isolado

O que define o tipo do teste é o **escopo exercitado**, o **grau de isolamento**, a **presença ou ausência de recursos externos reais** e o **nível arquitetural envolvido**.

Portanto, esta inferência é incorreta:

> “Se usei mock, então o teste é unitário.”

Ela é falsa.

O uso de doubles ajuda a controlar dependências, mas **não classifica sozinho o teste**.

11. Relação com testes de estado e testes de interação

Essa distinção também é muito importante.

11.1 Testes orientados a estado

Verificam o resultado final:

- retorno da função
- mudança de estado
- dado persistido

Esses testes costumam combinar bem com:

- **stub**
- **fake**

11.2 Testes orientados a interação

Verificam a colaboração entre objetos:

- se chamou
- quantas vezes chamou
- com que argumentos

Esses testes costumam combinar bem com:

- **spy**
- **mock**

Em termos didáticos, uma boa prática é priorizar, quando possível, testes mais centrados em comportamento observável relevante e evitar verificar interações desnecessárias.

12. Exemplo integrado com SUT e test doubles

Considere o caso de uso `ApproveTripRequest` , coerente com o domínio de gestão de viagens institucionais.

12.1 Dependências do caso de uso

```
interface TripRequestRepository {
  loadById(requestId: string): Promise<TripRequest | null>;
  save(tripRequest: TripRequest): Promise<void>;
}

interface NotificationService {
  sendApprovalEmail(email: string): Promise<void>;
}

class ApproveTripRequest {
  constructor(
    private readonly tripRequestRepository: TripRequestRepository,
    private readonly notificationService: NotificationService,
  ) {}

  async execute(requestId: string): Promise<void> {
    const tripRequest = await this.tripRequestRepository.loadById(requestId);

    if (tripRequest === null) {
      throw new Error("Trip request not found");
    }

    tripRequest.approve();
    await this.tripRequestRepository.save(tripRequest);
    await this.notificationService.sendApprovalEmail(
      tripRequest.requesterEmail,
    );
  }
}
```

Neste exemplo:

- o **SUT** é `ApproveTripRequest`
- os **DOCs** são:
  - `TripRequestRepository`
  - `NotificationService`

Dummy

Se o cenário falha antes de qualquer envio de e-mail, um `DummyNotificationService` pode ser suficiente.

Stub

Se você quer simular que a solicitação existe e está pendente, um `TripRequestRepositoryStub` pode devolver um objeto pronto.

Spy

Se você quer verificar se o e-mail de aprovação foi enviado, um `NotificationServiceSpy` pode registrar o endereço utilizado.

Mock

Se a expectativa central do teste é que `sendApprovalEmail()` **deve** ser chamado exatamente uma vez com determinado e-mail, você pode usar um mock configurado para isso.

Fake

Se você quer evitar um banco real, mas manter um comportamento mais natural de persistência, pode usar um `InMemoryTripRequestRepository`.

13. Exemplo explícito de cada double

13.1 Dummy

```
class DummyNotificationService implements NotificationService {
  async sendApprovalEmail(_email: string): Promise<void> {
    throw new Error("Should not be called");
  }
}
```

Neste trecho, o **test double** é `DummyNotificationService`.

13.2 Stub

```
class TripRequestRepositoryStub implements TripRequestRepository {
  async loadById(_requestId: string): Promise<TripRequest | null> {
    return TripRequest.pending("request-1", "professor@uespi.br");
  }

  async save(_tripRequest: TripRequest): Promise<void> {}
}
```

Neste trecho, o **test double** é `TripRequestRepositoryStub`.

O papel predominante dele é o de **stub**, porque fornece ao SUT a resposta necessária ao cenário.

13.3 Spy

```
class NotificationServiceSpy implements NotificationService {
  public readonly emails: string[] = [];

  async sendApprovalEmail(email: string): Promise<void> {
    this.emails.push(email);
  }
}
```

Neste trecho, o **test double** é `NotificationServiceSpy`.

O papel predominante dele é o de **spy**, porque registra o que aconteceu para verificação posterior.

13.4 Fake

```
class InMemoryTripRequestRepository implements TripRequestRepository {
  private readonly items: TripRequest[] = [];

  async loadById(requestId: string): Promise<TripRequest | null> {
    return this.items.find((item) => item.id === requestId) ?? null;
  }

  async save(tripRequest: TripRequest): Promise<void> {
    const index = this.items.findIndex((item) => item.id === tripRequest.id);

    if (index === -1) {
      this.items.push(tripRequest);
      return;
    }

    this.items[index] = tripRequest;
  }
}
```

Neste trecho, o **test double** é `InMemoryTripRequestRepository`.

O papel predominante dele é o de **fake**, porque implementa uma versão funcional simplificada do repositório.

14. Problema comum: chamar tudo de mock

No uso cotidiano de ferramentas como Jest, Vitest, Sinon e semelhantes, é comum chamar qualquer substituto de “mock”. Isso acontece porque as bibliotecas concentram múltiplas funcionalidades sob APIs de mocking.

Contudo, do ponto de vista conceitual, isso é impreciso.

Por exemplo, uma função simulada que apenas retorna um valor fixo está mais próxima de um **stub** do que de um mock propriamente dito.

Essa distinção importa porque melhora:

- a clareza da discussão técnica
- a escolha do double adequado
- a qualidade do desenho dos testes

15. Heurística prática para escolher

Uma regra prática bastante útil:

Use dummy

quando a dependência não importa no cenário

Use stub

quando você precisa controlar entradas indiretas

Use spy

quando quer observar efeitos indiretos

Use mock

quando a interação é requisito central do comportamento

Use fake

quando uma implementação simplificada é mais expressiva que retornos artificiais

16. Qual deles deve ser preferido?

Do ponto de vista de qualidade de testes, costuma ser saudável esta inclinação:

- preferir **fakes** e **stubs** quando possível
- usar **spies** quando a interação realmente importa
- usar **mocks** com parcimônia
- usar **dummies** apenas como preenchimento

A razão é que **mocks em excesso** frequentemente tornam os testes frágeis, muito presos aos detalhes internos da implementação.

Já **fakes bem construídos** e **stubs simples** tendem a gerar testes mais robustos, legíveis e compatíveis com refatoração.

17. Síntese final

Em formulação compacta:

- **Dummy**: só ocupa lugar
- **Stub**: devolve respostas
- **Spy**: registra chamadas
- **Mock**: verifica expectativas de interação
- **Fake**: implementação simplificada, mas funcional

Ou, em linguagem ainda mais intuitiva:

- **Dummy** → “figurante”
- **Stub** → “respondedor controlado”
- **Spy** → “observador”
- **Mock** → “fiscal de interação”
- **Fake** → “versão simplificada que funciona”

A pergunta mais útil, portanto, não é:

> “Qual é o maior ou o mais sofisticado?”

Mas sim:

> **“Que papel esta dependência substituta desempenha neste teste?”**

Essa é a pergunta que realmente ajuda a escolher o double adequado.